

Valider un modèle de classification

Les méthodes au-delà du Leave-One-Out

Cours synthétique appliqué au projet SVM — broyeur B430

Classification BON / MAUVAIS · 28 lots · classes 20 / 8 · 3 variables (Durée, Température, Pression)

Tu as évalué tes modèles uniquement en **Leave-One-Out Cross-Validation (LOOCV)**. C'est un choix défendable sur un petit jeu de données, mais ce n'est ni la seule méthode ni toujours la meilleure. Ce document présente les principales alternatives, leur principe, leurs forces et faiblesses, et — pour chacune — si elle est pertinente dans **ton** cas précis (28 lots, fort déséquilibre 20/8).

0. De quoi parle-t-on exactement ?

« Tester un modèle » veut dire **estimer la performance qu'il aurait sur des données jamais vues**. Le problème central : si tu évalues sur les mêmes données que celles d'entraînement, tu obtiens un score trop optimiste. Toutes les méthodes ci-dessous résolvent ça en séparant, d'une façon ou d'une autre, les données qui entraînent de celles qui évaluent.

Un bon estimateur de performance se juge sur deux axes. Le **biais** : sous-estime-t-il ou surestime-t-il la vraie performance ? La **variance** : si je refais l'expérience avec un découpage différent, le chiffre change-t-il beaucoup ? Sur un jeu de 28 points, la **variance de l'estimation est l'ennemi principal** : un seul lot mal classé fait bouger l'accuracy de ~3,6 points. Garde ça en tête, c'est le fil rouge de tout le cours.

1. Hold-out (séparation train / test)

La méthode la plus simple : on réserve une fraction des données (souvent 20–30 %) comme jeu de test, on entraîne sur le reste, on évalue une seule fois sur le test. C'est le `train_test_split` classique.

Avantages	Limites
+ Très rapide : un seul entraînement. + Simple à comprendre et à mettre en œuvre. + Convient quand les données sont nombreuses (milliers d'exemples).	— Estimation à très forte variance : le résultat dépend du tirage. — On « gaspille » des données : le modèle final n'a pas vu le test. — Sur 28 lots, un test de 20 % = 5–6 lots → score quasi aléatoire.

À surveiller — verdict : à éviter

Avec 28 lots, un hold-out unique donne une estimation tellement instable qu'elle n'est pas exploitable. C'est précisément la raison pour laquelle tu es passé à la validation croisée. Ne reviens pas en arrière sur ce point.

2. Validation croisée en k blocs (k-fold)

On découpe les données en **k** blocs (folds) de taille égale. À tour de rôle, chaque bloc sert de test pendant que les $k-1$ autres entraînent. On obtient k scores, dont on prend la moyenne. Valeurs usuelles : $k = 5$ ou $k = 10$.

Le **LOOCV** que tu utilises est le cas extrême $k = n$: 28 blocs d'un seul lot chacun. Comprendre k-fold, c'est donc resituer ta méthode dans une famille plus large.

Avantages	Limites
+ Bien meilleur compromis biais/variance que le hold-out. + Chaque lot sert exactement une fois en test. + $k=5$ ou $10 \rightarrow 5$ à 10 entraînements seulement (rapide).	— Le découpage aléatoire introduit un peu de hasard (atténué par la stratification, §3). — k-fold simple ne garantit pas la présence de MAUVAIS dans chaque test.

Pour ton projet B430 — verdict : utilisable, mais en version stratifiée (voir §3)

Un 5-fold ou 10-fold est une alternative légitime au LOOCV, plus rapide et souvent à variance d'estimation plus faible. Mais en k-fold *simple*, avec seulement 8 MAUVAIS, un fold pourrait n'en contenir aucun $\rightarrow$ score faussé. D'où l'étape suivante.

3. k-fold stratifié — le plus important pour toi

Variante de k-fold qui **conserve la proportion des classes dans chaque bloc**. Avec ton ratio 20 BON / 8 MAUVAIS (~71 % / 29 %), chaque fold reproduit approximativement cette répartition. C'est le comportement par défaut de scikit-learn pour la classification via `cross_val_score`.

Point clé souvent ignoré : le LOOCV ne peut pas être stratifié. Chaque test ne contient qu'un seul lot, donc impossible d'y respecter une proportion de classes. Pour un jeu déséquilibré comme le tien, c'est un vrai désavantage du LOOCV face au k-fold stratifié.

Avantages	Limites
+ Protège contre les folds « sans MAUVAIS », fréquents quand une classe est rare. + Estimation plus stable et plus juste sur données déséquilibrées. + Aucun coût supplémentaire par rapport au k-fold simple.	— Toujours un peu de hasard dans le découpage (régulé au §4 par la répétition). — Avec 8 MAUVAIS, $k=10$ mettrait <1 MAUVAIS par fold : préférer $k=4$ ou $k=5$.

Pour ton projet B430 — verdict : recommandé — ta référence de base

Un **StratifiedKFold** à $k=4$ ou $k=5$ est probablement le meilleur compromis simple pour le B430 : il respecte le déséquilibre, reste rapide, et chaque test contient 1–2 MAUVAIS. Code :

```
cross_val_score(model, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=0)).
```

4. k-fold stratifié répété

On répète le k-fold stratifié plusieurs fois avec des découpages aléatoires différents, puis on moyenne tous les scores. Exemple : 5 folds $\times$ 10 répétitions = 50 estimations. But : **noyer le hasard du découpage** et obtenir un chiffre stable et reproductible.

Avantages	Limites
+ Réduit fortement la variance de l'estimation — décisif sur petit jeu. + Donne une distribution de scores $\rightarrow$ tu peux citer une moyenne $\pm$ écart-type. + Toujours peu coûteux ici (50 entraînements = quelques secondes).	— Plus long que k-fold simple (mais négligeable à $n=28$). — Ne corrige pas le biais lié au réglage d'hyperparamètres (voir §6).

Pour ton projet B430 — verdict : très recommandé

RepeatedStratifiedKFold($n_splits=5$, $n_repeats=10$) est sans doute la meilleure évaluation « ordinaire » pour ton dataset. Plutôt qu'un seul chiffre LOOCV fragile, tu annonces par ex. « 83 % $\pm$ 4 % ». C'est plus honnête et plus convaincant dans un rapport.

5. Leave-One-Out (ce que tu as fait) — à sa juste place

LOOCV = k-fold avec $k = n$. Chacun des 28 lots est testé une fois, le modèle s'entraînant sur les 27 autres. Ce n'est pas « faux », mais il faut connaître ses limites réelles :

Avantages	Limites
+ Quasiment sans biais : on entraîne sur presque toutes les données ($n-1$). + Déterministe : aucun hasard de découpage, résultat reproductible. + Exploite au maximum un très petit jeu de données.	— Variance d'estimation élevée : les 28 modèles sont presque identiques et corrélés. — Impossible à stratifier $\rightarrow$ gênant avec ton déséquilibre 20/8. — Coûteux quand n grandit (28 entraînements ici, mais 28 000 sur gros jeu). — Score « tout ou rien » par lot : peu lisible en termes d'incertitude.

Pour ton projet B430 — verdict : garde-le, mais complète-le

Le LOOCV reste pertinent comme *une* mesure parmi d'autres. Mais ne le présente pas comme la vérité unique : compare-le au k-fold stratifié répété. Si les deux concordent, ta conclusion est solide ; s'ils divergent, c'est le signal que ton estimation est instable — une information précieuse en soi.

6. Validation croisée imbriquée (nested CV)

Indispensable dès qu'on **règle des hyperparamètres** (ce que fait ton GridSearchCV sur C et γ). Deux boucles imbriquées : la boucle *interne* choisit les meilleurs hyperparamètres, la boucle *externe* mesure la performance sur des données que ce réglage n'a jamais vues. On évite ainsi de « tester sur ce qui a servi à régler ».

À surveiller — À surveiller dans ton notebook actuel

Tu fais GridSearchCV (sections 8/8b) pour trouver C et gamma, **puis** tu rapportes l'accuracy LOOCV de ces paramètres optimisés. Or ces paramètres ont été choisis *en regardant* les mêmes données : le score affiché est donc **optimistement biaisé**. La nested CV est la correction propre. Sans elle, ton « 82,1 % » SVM RBF est probablement un peu surévalué.

Avantages	Limites
+ Seule méthode donnant une estimation non biaisée AVEC réglage d'hyperparamètres. + Reflète la vraie performance de ta procédure complète (réglage inclus). + Directement disponible : <code>cross_val_score</code> sur un objet GridSearchCV.	— Coûteux : c'est une CV dans une CV (mais gérable à $n=28$). — Plus complexe à expliquer et à mettre en place. — Ne fournit pas un unique jeu d'hyperparamètres « final » (c'est normal).

Pour ton projet B430 — verdict : recommandé pour annoncer un chiffre honnête

Dès que tu compares des modèles réglés par GridSearch, fais-le en nested CV. Schéma : `cross_val_score(GridSearchCV(SVC(), grille, cv=inner), X, y, cv=outer)`, avec inner et outer en StratifiedKFold. C'est ce chiffre-là qu'il faut mettre dans la conclusion.

7. Bootstrap (et .632)

Approche par rééchantillonnage : on tire B échantillons « bootstrap » de taille n **avec remise**. En moyenne ~63 % des lots sont tirés (entraînement), les ~37 % jamais tirés servent de test (« out-of-bag »). On répète B fois (ex. 1000) → une distribution de scores. La variante .632 corrige le biais en pondérant score OOB et score d'entraînement.

Avantages	Limites
+ Donne un véritable intervalle de confiance sur l'accuracy. + Exploite bien les petits jeux ; B très grand sans coût rédhibitoire ici. + Cadre statistique solide pour quantifier l'incertitude.	— Le tirage avec remise duplique des lots → peut sur-optimiser un peu. — Plus subtil à interpréter (.632 / .632+) que la simple CV. — Moins « standard » en présentation que le k-fold.

Pour ton projet B430 — verdict : optionnel mais élégant pour l'incertitude

Si tu veux accompagner ton accuracy d'un **intervalle de confiance** (« 82 %, IC95 % [71 %–93 %] »), le bootstrap est l'outil idéal. Ce n'est pas prioritaire, mais ça renforce la rigueur d'un rapport sur si peu de données.

8. Découpages aléatoires répétés (Monte-Carlo / ShuffleSplit)

On répète un hold-out aléatoire plusieurs fois (ex. 100 tirages 80/20 stratifiés) et on moyenne. Proche du k-fold répété, mais les jeux de test peuvent se chevaucher et un lot peut n'être jamais testé. StratifiedShuffleSplit en est la version respectant les classes.

Avantages	Limites
-----------	---------

+ On contrôle librement la taille du test et le nombre de répétitions.
+ Estimation lissée par la répétition.

— Couverture non garantie : certains lots jamais testés, d'autres souvent.
— Sur $n=28$, le k-fold stratifié répété (§4) est généralement préférable.

À surveiller — verdict : redondant ici

Apporte peu par rapport au k-fold stratifié répété, qui garantit en plus que chaque lot est testé. À connaître, mais pas la peine de l'ajouter à ton projet.

9. Deux pièges transverses, indépendants de la méthode

Aucune méthode de validation ne te sauvera de ces deux erreurs :

a) Fuite de données par la normalisation. Ton `StandardScaler` est ajusté sur l'ensemble du jeu *avant* la validation croisée (documenté dans ton `CLAUDE.md`). En toute rigueur, la moyenne et l'écart-type du scaler « voient » alors les lots de test → légère fuite, donc score un peu optimiste. La correction propre : mettre le scaler et le SVM dans un `Pipeline`, et passer ce `Pipeline` à la CV — ainsi le scaler est réajusté sur le seul pli d'entraînement à chaque tour.

b) L'accuracy est trompeuse à 20/8. Un modèle qui prédit « BON » pour tout atteint déjà 71,4 % (c'est exactement ce que font tes SVM linéaire/poly dégénérés). Donc « 71 % » ne veut rien dire ici. Regarde plutôt la **balanced accuracy**, le **F1** de la classe MAUVAIS, la **matrice de confusion** et le **ROC-AUC**. Sur un problème qualité, manquer un lot MAUVAIS (faux négatif) coûte bien plus cher qu'une fausse alerte — le rappel sur MAUVAIS est sans doute ta vraie métrique cible.

Le choix de la *métrique* est souvent plus déterminant que le choix de la *méthode de découpage*. Ne néglige pas ce point.

10. Tableau de synthèse

Méthode	Principe en une ligne	Variance estim.	Pertinent B430 ?
Hold-out	Un seul découpage train/test	Très élevée	Non — trop peu de données
k-fold	k blocs, chacun test à tour de rôle	Moyenne	Oui, mais en stratifié
k-fold stratifié	k-fold respectant le ratio 20/8	Moyenne	Oui — base recommandée (k=4–5)
Strat. répété	k-fold stratifié répété puis moyenné	Faible	Oui — meilleure éval. ordinaire
LOOCV	$k = n$ (ce que tu fais)	Élevée	Oui, mais à compléter
Nested CV	CV interne (réglage) + externe (mesure)	Faible	Oui — pour ton GridSearch
Bootstrap	Rééchantillonnage avec remise + OOB	Faible	Optionnel — intervalle de confiance
ShuffleSplit	Hold-outs aléatoires répétés	Faible	Redondant avec strat. répété

11. Recommandation concrète pour le B430

Si tu ne devais changer qu'une chose, voici l'ordre de priorité :

1 Mets ton scaler + ton modèle dans un `Pipeline` et passe-le à la CV (corrige la fuite de données).

- 2 Remplace/complète le LOOCV par un **RepeatedStratifiedKFold(5, 10)** et rapporte « moyenne $\pm$ écart-type ».
- 3 Pour comparer tes modèles réglés par GridSearch, utilise une **nested CV** : c'est le chiffre honnête à mettre en conclusion.
- 4 Change de métrique : **balanced accuracy + F1/rappel de la classe MAUVAIS + matrice de confusion**, pas l'accuracy brute.
- 5 Optionnel : un **bootstrap** pour donner un intervalle de confiance.

Squelette de code (à adapter)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import (RepeatedStratifiedKFold,
                                     cross_val_score, GridSearchCV, StratifiedKFold)

pipe = make_pipeline(StandardScaler(), SVC(class_weight='balanced'))

# Éval ordinaire, stable, sans fuite :
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0)
scores = cross_val_score(pipe, X_raw, y, cv=cv, scoring='balanced_accuracy')
print(scores.mean(), scores.std())

# Éval honnête AVEC réglage d'hyperparamètres (nested) :
grid = {'svc__C':[0.1,1,10], 'svc__gamma':[0.1,1,10]}
inner = StratifiedKFold(4, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)
clf = GridSearchCV(pipe, grid, cv=inner, scoring='balanced_accuracy')
nested = cross_val_score(clf, X_raw, y, cv=outer, scoring='balanced_accuracy')
print(nested.mean(), nested.std())
```

En résumé : le LOOCV n'était pas une erreur, mais c'est l'option la moins adaptée à un jeu *déséquilibré*. Le couple **k-fold stratifié répété** (pour mesurer) + **nested CV** (pour le réglage), avec une **métrique adaptée au déséquilibre**, te donnera des conclusions nettement plus solides et défendables.